

PySpark Data Engineering Interview Handbook

100 real-world questions and answers for Data Engineer, Senior Data Engineer, Data Architect, AI/ML Data Engineering and Lakehouse roles

Designed for interviews involving Databricks, Apache Spark, Azure Databricks, AWS EMR/Glue, Synapse Spark, Delta Lake, Kafka, SQL Server, PostgreSQL, Oracle, S3, ADLS, Athena, Hive and enterprise banking data platforms.

How to Use This Handbook

This handbook is written for practical interview preparation. The questions are based on common real-world patterns asked in Data Engineering interviews: large file processing, Spark performance, joins, skew, Delta Lake, streaming, CDC, data quality, production deployment, banking scenarios and AI/ML data pipelines.

Important note: companies rarely publish exact proprietary interview questions. The questions here represent realistic patterns frequently tested by top employers and interview panels.

Reference Schemas

Reference schemas used in examples:

- banking.transactions(account_id, customer_id, txn_id, txn_ts, amount, merchant, channel, status, ingestion_ts)
- banking.accounts(account_id, customer_id, product_type, open_date, close_date, balance, status)
- banking.customer_dim(customer_id, name, segment, risk_band, effective_start, effective_end, is_current)
- ecommerce.orders(order_id, customer_id, order_ts, amount, status, updated_at)
- insurance.claims(claim_id, policy_id, claim_ts, amount, claim_status, fraud_score)
- lake bronze/silver/gold paths in ADLS/S3 or Databricks Delta tables

PySpark Fundamentals and DataFrame API

Q1. How do you explain Spark architecture in a data engineering interview?

Real answer: Spark uses a driver program to build a logical plan and executors to run tasks in parallel across partitions. In production, the driver coordinates jobs, stages and tasks, while executors process data from storage such as S3, ADLS, HDFS, Kafka or Delta Lake. A strong answer must mention lazy evaluation, DAG optimization, shuffle boundaries, executor memory, partitioning and cluster manager such as YARN, Kubernetes or Databricks.

Code / SQL / PySpark example:

```
df = spark.read.format('delta').table('silver.transactions')
print(df.rdd.getNumPartitions())
df.groupBy('channel').count().explain(True)
```

Real database / enterprise context: Databricks, EMR, Synapse Spark and on-prem Hadoop all use this model; storage and cluster manager vary.

Interview tip: Do not say Spark is just faster Python. Explain driver, executor, partition, shuffle and DAG.

Q2. What is lazy evaluation in PySpark and why does it matter?

Real answer: Transformations like select, filter and join are lazy. Spark builds a logical plan and only executes it when an action such as count, collect, write or show is called. This allows Catalyst to optimize the plan before execution. In interviews, explain that lazy evaluation reduces unnecessary work but can hide errors until the action runs.

Code / SQL / PySpark example:

```
df = spark.table('bronze.transactions')
filtered = df.filter('amount > 1000') # no execution yet
filtered.write.mode('overwrite').saveAsTable('silver.high_value_txn') # action
```

Real database / enterprise context: Seen in all Spark databases: Delta, Hive Metastore, Glue Catalog and Unity Catalog.

Interview tip: Mention that count() for debugging can be expensive on large datasets.

Q3. Difference between DataFrame, Dataset and RDD?

Real answer: RDD is the low-level distributed collection API. DataFrame is a distributed table with named columns and Catalyst optimization. Dataset is a typed API mainly used in Scala/Java, not Python. For PySpark data engineering, DataFrames are preferred because they are optimized, expressive and integrate with SQL, Delta and catalog tables.

Code / SQL / PySpark example:

```
rdd = spark.sparkContext.parallelize([('A', 10), ('B', 20)])
df = rdd.toDF(['category', 'amount'])
df.groupBy('category').sum('amount').show()
```

Real database / enterprise context: Enterprise pipelines typically use DataFrames and Spark SQL, not raw RDDs.

Interview tip: Use RDD only for low-level custom processing; avoid it for standard ETL.

Q4. How do you read CSV, JSON, Parquet and Delta files in PySpark?

Real answer: Use explicit schema for CSV/JSON where possible, because schema inference causes an extra scan and may infer wrong types. Prefer Parquet or Delta for production because they are columnar, compressed and support predicate pushdown. Delta adds ACID transactions and schema evolution.

Code / SQL / PySpark example:

```
from pyspark.sql.types import *
schema = StructType([StructField('account_id', StringType()), StructField('amount', DoubleType())])
df = spark.read.option('header', True).schema(schema).csv('/mnt/raw/transactions/')
parq = spark.read.parquet('/mnt/silver/transactions/')
delta = spark.read.format('delta').load('/mnt/gold/transactions_delta/')
```

Real database / enterprise context: PostgreSQL/Oracle extracts often land as CSV initially; lakehouse zones should use Parquet/Delta.

Interview tip: Always discuss schema control, corrupt records and data quality when reading raw files.

Q5. How do you define and enforce schema in PySpark?

Real answer: Define StructType schemas explicitly for raw ingestion. This prevents silent type drift, inconsistent nullability and bad inference. In regulated banking pipelines, schema should be versioned and validated before loading into silver or gold tables.

Code / SQL / PySpark example:

```
from pyspark.sql.types import StructType, StructField, StringType, TimestampType, DecimalType
```

```

schema = StructType([
    StructField('txn_id', StringType(), False),
    StructField('account_id', StringType(), False),
    StructField('txn_ts', TimestampType(), True),
    StructField('amount', DecimalType(18,2), True)
])
df = spark.read.schema(schema).json('/mnt/bronze/txn/')

```

Real database / enterprise context: Works with Databricks Auto Loader, AWS Glue catalog tables and Hive external tables.

Interview tip: A mature answer includes schema drift handling and quarantine for bad records.

Q6. How do you handle corrupt records during ingestion?

Real answer: Use permissive mode with a corrupt record column or bad records path. Good production design separates valid records from rejected records and logs rejection counts. Do not simply drop bad records in financial systems because auditability is required.

Code / SQL / PySpark example:

```

df = (spark.read
    .option('mode', 'PERMISSIVE')
    .option('columnNameOfCorruptRecord', '_corrupt_record')
    .json('/mnt/raw/customer_events'))
valid = df.filter('_corrupt_record IS NULL')
bad = df.filter('_corrupt_record IS NOT NULL')

```

Real database / enterprise context: In banking, bad records should be written to an exception table with batch_id and ingestion_ts.

Interview tip: Mention replay capability: after fixing source issue, rejected rows can be reprocessed.

Q7. How do you select, rename and cast columns safely?

Real answer: Use select with aliases and explicit casts. Avoid select * in production because upstream schema changes can break downstream models or introduce unexpected columns. Casting should include validation because invalid values become null in many Spark operations.

Code / SQL / PySpark example:

```

from pyspark.sql.functions import col
df2 = df.select(
    col('customer_id').cast('string'),
    col('amount').cast('decimal(18,2)').alias('txn_amount'),
    col('txn_ts').cast('timestamp')
)

```

Real database / enterprise context: Used when converting raw Oracle/SQL Server exports into silver Delta tables.

Interview tip: Interviewers like hearing 'contract-based schema' and 'no select star in curated layers'.

Q8. What is the difference between withColumn and select for adding columns?

Real answer: withColumn is convenient for adding or replacing one column. However, chaining many withColumn calls can create long logical plans. For multiple derived columns, select is often cleaner and can reduce plan complexity.

Code / SQL / PySpark example:

```

from pyspark.sql.functions import col, when
# Better for many expressions
df2 = df.select('*',
    (col('amount') * 0.2).alias('vat'),
    when(col('amount') > 1000, 'HIGH').otherwise('NORMAL').alias('txn_band')
)

```

Real database / enterprise context: Common in transaction enrichment, customer segmentation and claims scoring.

Interview tip: Say you check explain() when many transformations make the plan too large.

Q9. How do you filter nulls, blanks and invalid values?

Real answer: Use isNull/isNotNull and trim for string fields. In source systems, blanks and nulls are often different. A robust pipeline normalizes empty strings, validates required fields and writes invalid rows to an exception dataset.

Code / SQL / PySpark example:

```

from pyspark.sql.functions import col, trim
valid = df.filter(col('account_id').isNotNull() & (trim(col('account_id')) != ''))
invalid = df.subtract(valid)

```

Real database / enterprise context: Banking customer/account feeds often contain blanks from legacy mainframe exports.

Interview tip: Do not use `df.na.drop()` blindly; it can remove useful records.

Q10. How do you remove duplicates in PySpark?

Real answer: `dropDuplicates` works for simple exact duplicates. For real-world deduplication, use window functions to keep the latest record based on update timestamp, ingestion timestamp or source priority. This is frequently asked because production pipelines often reprocess files or receive duplicate CDC events.

Code / SQL / PySpark example:

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, col
w = Window.partitionBy('customer_id').orderBy(col('updated_at').desc(), col('ingestion_ts').desc())
deduped = df.withColumn('rn', row_number().over(w)).filter('rn = 1').drop('rn')
```

Real database / enterprise context: PostgreSQL CDC, Kafka topics, S3 landing zones and Delta bronze tables often need this pattern.

Interview tip: Always clarify dedup key and ordering rule before writing code.

Q11. How do you handle missing values for analytics and ML pipelines?

Real answer: For data engineering, missing values are handled based on business meaning. For analytics, you may default unknown categories. For ML, imputation must avoid data leakage. For regulated reporting, nulls may need to remain visible with quality flags rather than being filled.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import coalesce, lit
df2 = df.withColumn('segment', coalesce('segment', lit('UNKNOWN')))
df2 = df2.withColumn('balance_missing_flag', col('balance').isNull())
```

Real database / enterprise context: In credit risk, missing income should not be blindly replaced without a model governance decision.

Interview tip: Strong candidates explain business semantics, not just `fillna()`.

Q12. How do you parse dates and timestamps from inconsistent source formats?

Real answer: Use `to_date/to_timestamp` with explicit formats and create exception handling for invalid values. If multiple formats exist, use `coalesce` across parsed formats. For streaming or cross-region data, timezone handling must be clearly defined.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import to_timestamp, coalesce, col
df2 = df.withColumn('txn_ts_parsed', coalesce(
    to_timestamp('txn_ts', 'yyyy-MM-dd HH:mm:ss'),
    to_timestamp('txn_ts', 'dd/MM/yyyy HH:mm')
))
invalid = df2.filter(col('txn_ts_parsed').isNull())
```

Real database / enterprise context: Athena/Presto, SQL Server and mainframe feeds often use different date formats.

Interview tip: Mention UTC standardization for event time in global systems.

Q13. How do you convert nested JSON into a flat table?

Real answer: Use `select` on nested fields and `explode` arrays. The important part is preserving record identity when exploding arrays. For semi-structured data, flatten only what is required for downstream analytics; keep raw JSON in bronze for replay.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import explode, col
flat = events.select(
    col('event_id'),
    col('customer.id').alias('customer_id'),
    explode('items').alias('item')
).select('event_id', 'customer_id', col('item.sku').alias('sku'), col('item.price').alias('price'))
```

Real database / enterprise context: Common with API, Kafka, MongoDB and event-tracking payloads.

Interview tip: Do not lose parent keys while exploding child arrays.

Q14. How do you join DataFrames and avoid ambiguous columns?

Real answer: Alias DataFrames and explicitly select required columns after join. Ambiguous columns are common when both DataFrames contain `customer_id`, `updated_at` or `status`. For maintainability, define join keys and output schema clearly.

Code / SQL / PySpark example:

```
t = transactions.alias('t')
a = accounts.alias('a')
joined = t.join(a, t.account_id == a.account_id, 'left')
result = joined.select('t.txn_id', 't.account_id', 'a.customer_id', 't.amount', 'a.product_type')
```

Real database / enterprise context: Used in customer 360 and transaction enrichment from core banking systems.

Interview tip: Avoid selecting all columns after joins in curated layers.

Q15. Difference between inner, left, right, full and semi joins in PySpark?

Real answer: Inner returns matching rows. Left keeps all left rows. Full keeps rows from both sides. Left semi returns left rows that have matches but does not include right columns. Left anti returns left rows with no matches. Semi and anti joins are heavily used in validation and delta detection.

Code / SQL / PySpark example:

```
new_accounts = source.join(target, 'account_id', 'left_anti')
existing_accounts = source.join(target, 'account_id', 'left_semi')
```

Real database / enterprise context: Left anti is common for identifying missing target records after migration.

Interview tip: Mention semi/anti joins; many candidates forget them.

Q16. How do you use groupBy and aggregation correctly?

Real answer: Use groupBy with aggregation functions and avoid collecting large results to the driver. In real pipelines, aggregations often trigger shuffles, so partitioning and grouping key cardinality matter. For financial metrics, cast decimals correctly before aggregation.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import sum, countDistinct
agg = df.groupBy('customer_id').agg(
    sum('amount').alias('total_amount'),
    countDistinct('txn_id').alias('txn_count')
)
```

Real database / enterprise context: Banking monthly spend, insurance claim totals and e-commerce revenue rollups use this pattern.

Interview tip: Explain shuffle cost when grouping by high-cardinality keys.

Q17. How do you use window functions for interview scenarios?

Real answer: Window functions solve ranking, deduplication, running totals, previous value comparison and latest record problems. They are core to PySpark data engineering interviews because they map directly to production ETL logic.

Code / SQL / PySpark example:

```
from pyspark.sql.window import Window
from pyspark.sql.functions import lag, sum as fsum
w = Window.partitionBy('account_id').orderBy('txn_ts')
df2 = df.withColumn('prev_amount', lag('amount').over(w))
df2 = df2.withColumn('running_balance', fsum('amount').over(w))
```

Real database / enterprise context: Useful for account balances, delinquency cycles, customer journeys and fraud patterns.

Interview tip: Know row_number, rank, dense_rank, lag, lead and running sums.

Q18. How do you union two DataFrames with different column order?

Real answer: Use unionByName instead of union. If schemas differ, add missing columns with nulls before unioning or use allowMissingColumns where supported. This prevents silent data corruption caused by positional column matching.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import lit
combined = df_2024.unionByName(df_2025, allowMissingColumns=True)
```

Real database / enterprise context: Common when appending monthly files from S3/ADLS where schema evolves over time.

Interview tip: Mention schema evolution and contract testing.

Q19. How do you generate surrogate keys in PySpark?

Real answer: Use deterministic hash keys for distributed data when possible. monotonically_increasing_id is unique but not consecutive and not deterministic across reloads. For dimensions, hash natural keys plus effective dates or use database-generated keys depending on architecture.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import sha2, concat_ws
dim = df.withColumn('customer_sk', sha2(concat_ws('|', 'customer_id', 'effective_start'), 256))
```

Real database / enterprise context: Lakehouse SCD2 dimensions often use hash-based surrogate keys.

Interview tip: Do not present monotonically_increasing_id as a universal surrogate key.

Q20. What is cache vs persist and when should you use them?

Real answer: cache stores a DataFrame in memory using default storage level. persist allows different storage levels such as MEMORY_AND_DISK. Use them when a DataFrame is reused multiple times and recomputation is expensive. Do not cache everything; it can evict useful data and cause memory pressure.

Code / SQL / PySpark example:

```
base = spark.table('silver.transactions').filter('txn_date >= date_sub(current_date(), 30)').persist()
base.groupBy('channel').count().show()
base.groupBy('status').count().show()
base.unpersist()
```

Real database / enterprise context: Used in feature engineering where the same cleaned dataset feeds multiple aggregations.

Interview tip: Mention unpersist() and memory trade-offs.

Performance and Optimization

Q21. How do you identify a slow PySpark job?

Real answer: Start with Spark UI: job duration, stage DAG, shuffle read/write, spill, task skew, executor failures and GC time. Then inspect explain plans for unnecessary shuffles, scans, Cartesian joins or missing predicate pushdown.

Code / SQL / PySpark example:

```
df.explain("formatted")
# Check Spark UI: SQL tab, Stages tab, Storage tab
```

Real database / enterprise context: Databricks, EMR and Synapse all expose Spark UI or equivalent job diagnostics.

Interview tip: Do not guess. Interviewers expect a systematic debug approach.

Q22. What is a shuffle and why is it expensive?

Real answer: A shuffle redistributes data across executors. It happens during joins, groupBy, distinct, repartition and window operations. It is expensive because data is serialized, written to disk/network and read by other tasks.

Code / SQL / PySpark example:

```
df.groupBy("customer_id").count() # likely shuffle
```

Real database / enterprise context: High-cardinality banking transaction aggregations can create large shuffle volumes.

Interview tip: Mention reducing data before shuffle and partitioning by join/group keys.

Q23. How do you fix data skew in PySpark?

Real answer: Data skew happens when a few keys hold disproportionate records, causing some tasks to run much longer. Fixes include salting hot keys, broadcast joins for small tables, skew join hints, pre-aggregation and better partitioning.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import concat, lit, floor, rand
skewed = df.withColumn("salt", floor(rand()*10))
```

Real database / enterprise context: A merchant like AMAZON or a default customer_id can dominate transaction volume.

Interview tip: Always identify skew from Spark UI before applying fixes.

Q24. When would you use broadcast join?

Real answer: Use broadcast join when one side is small enough to fit in executor memory. It avoids shuffling the large table. In production, broadcast reference dimensions such as currency, country, product type or risk code tables.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import broadcast
result = fact.join(broadcast(dim_product), "product_id", "left")
```

Real database / enterprise context: Databricks and Spark SQL can auto-broadcast based on autoBroadcastJoinThreshold.

Interview tip: Do not broadcast a table just because it has fewer rows; consider memory size.

Q25. How do you tune number of partitions?

Real answer: Partition count should match data size, cluster cores and downstream file sizing. Too few partitions underutilize cluster; too many create scheduling overhead and small files. A common target for Parquet/Delta output is roughly 128MB-1GB per file depending on workload.

Code / SQL / PySpark example:

```
df = df.repartition(200, "account_id")
df.write.mode("overwrite").partitionBy("month_end_date").parquet(path)
```

Real database / enterprise context: Useful in S3/ADLS lakehouse tables queried by Athena, Databricks SQL or Synapse.

Interview tip: Mention avoiding both huge files and small file problem.

Q26. Difference between repartition and coalesce?

Real answer: repartition performs a full shuffle and can increase or decrease partitions. coalesce reduces partitions with less movement and is useful before writing smaller outputs. Use repartition when changing partitioning key; use coalesce when reducing partitions after filtering.

Code / SQL / PySpark example:

```
small = df.filter("month_end_date = '2025-01-31'").coalesce(20)
by_customer = df.repartition("customer_id")
```

Real database / enterprise context: Common when writing monthly regulatory extracts.

Interview tip: Do not coalesce to 1 for large data unless producing a small control file.

Q27. How do you avoid small file problems?

Real answer: Control output partitions, use Delta OPTIMIZE/compaction, auto compaction where available and avoid excessive partitionBy columns. Small files hurt metadata listing and query planning on cloud object stores.

Code / SQL / PySpark example:

```
spark.sql("OPTIMIZE gold.transactions ZORDER BY (customer_id)")
```

Real database / enterprise context: Delta Lake on Databricks handles this better than raw Parquet on S3/ADLS.

Interview tip: Mention file size, metadata overhead and downstream query performance.

Q28. What is predicate pushdown?

Real answer: Predicate pushdown means filters are pushed to the data source so Spark reads less data. It works well with columnar formats like Parquet/Delta and partitioned data. Use filters early and avoid wrapping partition columns in functions.

Code / SQL / PySpark example:

```
df = spark.read.parquet(path).filter("txn_date >= '2025-01-01'")
```

Real database / enterprise context: Athena, Snowflake external tables and Databricks all benefit from pruning/pushdown.

Interview tip: Explain that where you place filters and how columns are stored matters.

Q29. How do you optimize joins in PySpark?

Real answer: Filter early, select only required columns, broadcast small tables, partition large tables by join key, handle skew and check join strategy in explain plan. Also consider bucketing/Z-order for repeated joins depending on platform.

Code / SQL / PySpark example:

```
left = fact.select("account_id", "amount", "txn_date")
right = dim.select("account_id", "customer_id")
joined = left.join(right, "account_id", "inner")
```

Real database / enterprise context: Banking transaction fact joined with account/customer dimension is a classic case.

Interview tip: A good answer mentions both code and physical execution.

Q30. How do you optimize PySpark UDFs?

Real answer: Avoid Python UDFs where built-in Spark SQL functions exist because Python UDFs require serialization between JVM and Python. Use built-ins, SQL expressions, pandas UDFs where appropriate, or Scala UDFs for performance-critical logic.

Code / SQL / PySpark example:


```
from pyspark.sql.functions import upper
df2 = df.withColumn("merchant_upper", upper("merchant"))
```

Real database / enterprise context: Used in cleansing merchant names, customer names and product codes.

Interview tip: Mention UDFs can block Catalyst optimization.

Delta Lake and Lakehouse

Q31. What is Delta Lake and why is it used in data engineering?

Real answer: Delta Lake adds ACID transactions, schema enforcement, time travel, merge/update/delete and reliable streaming/batch support on data lakes. It solves problems of raw Parquet lakes: partial writes, concurrent writes and difficult updates.

Code / SQL / PySpark example:

```
df.write.format("delta").mode("append").saveAsTable("silver.transactions")
```

Real database / enterprise context: Databricks Delta, Microsoft Fabric and open-source Delta all use transaction logs.

Interview tip: Explain `_delta_log` and ACID clearly.

Q32. How do you perform upsert using Delta MERGE?

Real answer: Use MERGE INTO to update existing rows and insert new rows based on keys. For CDC, apply latest event per key before merge to avoid multiple source matches.

Code / SQL / PySpark example:

```
MERGE INTO silver.accounts t
USING updates s
ON t.account_id = s.account_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Real database / enterprise context: Used for customer/account master tables from Oracle, SQL Server or Kafka CDC.

Interview tip: Mention deduplicating source updates before merge.

Q33. How do you implement SCD Type 2 in PySpark/Delta?

Real answer: Detect changed attributes using hash comparison, expire current records by setting `effective_end` and `is_current=false`, then insert new current records. Use Delta MERGE carefully, often with staged updates.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import sha2, concat_ws
stg = src.withColumn("hash", sha2(concat_ws("|", "name", "risk_band"), 256))
# expire old rows and insert new version using Delta merge
```

Real database / enterprise context: Customer risk band changes in banking require history for audit and reporting.

Interview tip: Explain `effective_start`, `effective_end`, `is_current` and hash diff.

Q34. How do you use Delta time travel?

Real answer: Delta time travel lets you query a table as of a version or timestamp, useful for debugging, rollback and audit. Retention settings affect how far back you can query.

Code / SQL / PySpark example:

```
old = spark.read.format("delta").option("versionAsOf", 42).table("gold.customer_dim")
```

Real database / enterprise context: Helpful when a production release corrupts a reporting table.

Interview tip: Mention VACUUM retention before promising unlimited history.

Q35. What is VACUUM in Delta Lake?

Real answer: VACUUM removes obsolete data files no longer needed by the Delta log after retention. It reduces storage cost but can remove files needed for time travel if retention is too aggressive.

Code / SQL / PySpark example:

```
spark.sql("VACUUM silver.transactions RETAIN 168 HOURS")
```

Real database / enterprise context: Storage cost control on ADLS/S3 lakehouse tables.

Interview tip: Do not run VACUUM 0 HOURS in production without understanding safety checks.

Q36. What is schema evolution in Delta?

Real answer: Schema evolution allows adding new columns during writes when enabled. It is useful for evolving source systems but should be governed. Schema enforcement prevents incompatible writes.

Code / SQL / PySpark example:

```
df.write.option("mergeSchema", "true").format("delta").mode("append").saveAsTable("bronze.events")
```

Real database / enterprise context: API payloads and Kafka events frequently add fields over time.

Interview tip: Say schema evolution should be allowed in bronze more than gold.

Q37. Bronze, Silver, Gold: how would you design a PySpark lakehouse?

Real answer: Bronze stores raw immutable data with metadata. Silver stores cleaned, typed, deduplicated and conformed data. Gold stores business-ready aggregates or dimensional models. Each layer should have tests, ownership and lineage.

Code / SQL / PySpark example:

```
bronze = spark.read.json(raw_path)
silver = bronze.filter("_corrupt_record IS NULL")
gold = silver.groupBy("customer_id").sum("amount")
```

Real database / enterprise context: Databricks, Fabric, Synapse and EMR lakehouses use similar medallion patterns.

Interview tip: Do not describe layers only; explain quality gates between layers.

Streaming and CDC

Q38. How do you read from Kafka using PySpark Structured Streaming?

Real answer: Use readStream with Kafka bootstrap servers and topic, parse key/value as strings or JSON, then write to Delta with checkpointing. Checkpointing is mandatory for fault tolerance.

Code / SQL / PySpark example:

```
raw = (spark.readStream.format("kafka")
      .option("kafka.bootstrap.servers", servers)
      .option("subscribe", "transactions")
      .load())
```

Real database / enterprise context: Kafka is common for card transactions, clickstream, fraud events and application logs.

Interview tip: Mention offsets, checkpoints, event time and schema parsing.

Q39. What is checkpointing in Structured Streaming?

Real answer: Checkpointing stores progress metadata and state so streaming queries can recover after failure without reprocessing everything. It is essential for exactly-once semantics with supported sinks like Delta.

Code / SQL / PySpark example:

```
query = df.writeStream.format("delta")
      .option("checkpointLocation", "/chk/txn")
      .start("/delta/txn")
```

Real database / enterprise context: On S3/ADLS, checkpoints must be stored reliably and separately per query.

Interview tip: Never reuse the same checkpoint for different query logic.

Q40. How do you handle late-arriving events in Spark streaming?

Real answer: Use event-time processing with watermarking. A watermark tells Spark how long to wait for late data before finalizing state. The delay depends on business SLA and event lateness patterns.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import window
agg = events.withWatermark("event_ts", "15 minutes")
      .groupBy(window("event_ts", "5 minutes"), "account_id").count()
```

Real database / enterprise context: Fraud and payment events may arrive late due to network or upstream delays.

Interview tip: Explain trade-off: longer watermark means more accuracy but more state and cost.

Q41. How do you process CDC data in PySpark?

Real answer: CDC data usually contains operation type, before/after values and commit timestamp. The pipeline should deduplicate by primary key and latest sequence, then apply inserts, updates and deletes using Delta MERGE.

Code / SQL / PySpark example:

```
w = Window.partitionBy("account_id").orderBy(col("commit_ts").desc())
latest = cdc.withColumn("rn", row_number().over(w)).filter("rn=1")
```

Real database / enterprise context: Sources include SQL Server CDC, Oracle GoldenGate, Debezium Kafka and AWS DMS.

Interview tip: Ask about operation codes: I/U/D and ordering column.

Q42. How do you make streaming writes idempotent?

Real answer: Use deterministic keys, checkpoints, foreachBatch with MERGE, and batch identifiers where needed. Idempotency means rerunning the same batch does not create duplicates or inconsistent state.

Code / SQL / PySpark example:

```
def upsert_batch(batch_df, batch_id):
    batch_df.createOrReplaceTempView("updates")
    spark.sql("MERGE INTO silver.txn t USING updates s ON t.txn_id=s.txn_id WHEN NOT MATCHED THEN INSERT *")
    stream.writeStream.foreachBatch(upsert_batch).option("checkpointLocation", chk).start()
```

Real database / enterprise context: Critical for banking payments and transaction pipelines.

Interview tip: Interviewers like hearing replay-safe and duplicate-safe design.

Data Quality and Testing

Q43. How do you perform data quality checks in PySpark?

Real answer: Create checks for nulls, uniqueness, referential integrity, accepted values, ranges and freshness. Write results to an audit table and fail or quarantine based on severity.

Code / SQL / PySpark example:

```
checks = df.agg(
    count("txn_id").alias("row_count"),
    sum(col("account_id").isNull().cast("int")).alias("null_account_count")
)
```

Real database / enterprise context: Regulatory reporting needs auditable data quality results.

Interview tip: Mention Great Expectations, Deequ or custom checks if relevant.

Q44. How do you reconcile source and target counts?

Real answer: Compare row counts, distinct key counts and control totals such as sum(amount) by business date or partition. Exact row counts alone are not enough because duplicates can hide missing records.

Code / SQL / PySpark example:

```
src = spark.table("src.txn").groupBy("month_end").agg(count("*").alias("src_count"), sum("amount").alias("src_amt"))
tgt = spark.table("gold.txn").groupBy("month_end").agg(count("*").alias("tgt_count"), sum("amount").alias("tgt_amt"))
recon = src.join(tgt, "month_end")
```

Real database / enterprise context: Used after migration from Hadoop/Hive to Databricks or AWS Athena.

Interview tip: Mention tolerance for decimals and time windows.

Q45. How do you write PySpark unit tests?

Real answer: Separate transformation logic into functions, build small input DataFrames, compare expected DataFrames and include edge cases. For production, combine unit tests with integration and data quality tests.

Code / SQL / PySpark example:

```
def active_accounts(df):
    return df.filter("status = 'ACTIVE'")
# pytest creates spark fixture, input_df, expected_df
```

Real database / enterprise context: CI/CD pipelines for Databricks repos or EMR jobs can run these tests.

Interview tip: Do not test only happy path; include nulls, duplicates and invalid data.

Q46. How do you validate schema changes before production?

Real answer: Compare incoming schema against expected contract. Classify changes as additive, breaking or type changes. Allow additive fields in bronze but require approvals for curated silver/gold schemas.

Code / SQL / PySpark example:

```
expected = set(["txn_id", "account_id", "amount"])
actual = set(df.columns)
missing = expected - actual
if missing: raise Exception(f"Missing columns: {missing}")
```

Real database / enterprise context: Prevents upstream Oracle/API changes from breaking reporting tables.

Interview tip: Mention contract testing and alerting.

Q47. How do you handle duplicate files in ingestion?

Real answer: Use file metadata, checksum, manifest tables or Auto Loader file tracking. Store processed file names and batch ids. This prevents loading the same source file multiple times.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import input_file_name
df = spark.read.parquet(path).withColumn("source_file", input_file_name())
```

Real database / enterprise context: Daily SFTP/S3/ADLS loads often resend files.

Interview tip: Idempotent file ingestion is a common senior interview topic.

Real-World Transformations

Q48. How would you calculate monthly account balance from transactions?

Real answer: Aggregate transactions by account and month, then join with opening balance or previous month balance. Use date truncation and window functions for running balance where required.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import last_day, sum as fsum
monthly = txn.withColumn("month_end", last_day("txn_ts"))
.groupBy("account_id", "month_end").agg(fsum("amount").alias("net_amount"))
```

Real database / enterprise context: Banking current account, savings and credit card ledgers.

Interview tip: Clarify if amounts are debits/credits and whether reversals exist.

Q49. How would you identify inactive customers?

Real answer: Define inactivity based on business rule, for example no successful transaction in 90 days. Use max transaction timestamp by customer and compare with processing date.

Code / SQL / PySpark example:

```
last_txn = txn.filter("status = 'SUCCESS']").groupBy("customer_id").agg(max("txn_ts").alias("last_txn_ts"))
inactive = last_txn.filter("last_txn_ts < date_sub(current_date(), 90)")
```

Real database / enterprise context: Used in customer retention, AML monitoring and product engagement.

Interview tip: Always ask what counts as activity.

Q50. How would you detect suspicious high-value transactions?

Real answer: Filter by threshold, velocity and historical behavior. A practical feature pipeline calculates rolling sums/counts and flags deviations. Rules are usually combined with ML scores and human review.

Code / SQL / PySpark example:

```
w = Window.partitionBy("account_id").orderBy("txn_ts").rangeBetween(-86400, 0)
# In practice convert timestamp to long seconds before rangeBetween
```

Real database / enterprise context: Fraud detection in banking and fintech.

Interview tip: Mention rule-based + ML + analyst workflow, not only threshold.

Q51. How do you build customer 360 in PySpark?

Real answer: Integrate customer profile, accounts, transactions, complaints, digital interactions and risk attributes into a conformed gold table. Use stable keys, deduplication, SCD2 dimensions and clear refresh strategy.

Code / SQL / PySpark example:

```
cust360 = customer.join(accounts_agg, "customer_id", "left").join(txn_agg, "customer_id", "left")
```

Real database / enterprise context: Used by banking CRM, marketing, risk and service teams.

Interview tip: Discuss identity resolution and data ownership.

Q52. How do you handle multi-currency transactions?

Real answer: Join transactions with FX rate table using currency and rate date. For historical reporting, use the rate valid on transaction date, not current rate. Store original and converted amount.

Code / SQL / PySpark example:

```
fxed = txn.join(fx, (txn.currency==fx.currency) & (to_date(txn.txn_ts)==fx.rate_date), "left")
fxed = fxed.withColumn("amount_gbp", col("amount") * col("gbp_rate"))
```

Real database / enterprise context: Banking, payments and global e-commerce.

Interview tip: Mention missing rates and audit requirements.

Q53. How do you calculate rolling 7-day transaction count?

Real answer: Use window functions with event timestamp converted to numeric seconds or use grouped daily aggregates then rolling window. For very large datasets, pre-aggregate first to reduce shuffle.

Code / SQL / PySpark example:

```
daily = txn.groupBy("account_id", to_date("txn_ts").alias("dt")).count()
w = Window.partitionBy("account_id").orderBy("dt").rowsBetween(-6,0)
out = daily.withColumn("rolling_7d_count", sum("count").over(w))
```

Real database / enterprise context: Fraud velocity checks and digital engagement metrics.

Interview tip: Explain row vs range window difference.

Q54. How do you build a top-N merchants report?

Real answer: Aggregate spend by customer and merchant, then use row_number over customer partition ordered by amount descending. This is a common interview question combining groupBy and window.

Code / SQL / PySpark example:

```
spend = txn.groupBy("customer_id", "merchant").agg(sum("amount").alias("spend"))
w = Window.partitionBy("customer_id").orderBy(col("spend").desc())
top3 = spend.withColumn("rn", row_number().over(w)).filter("rn <= 3")
```

Real database / enterprise context: Used in personalization and card spend analytics.

Interview tip: Mention tie handling: row_number vs rank.

Q55. How do you compare two large DataFrames?

Real answer: Compare schema, row counts, key counts, checksums/hash totals and mismatched rows using anti joins. For large datasets, compare by partition or business date to isolate issues.

Code / SQL / PySpark example:

```
left_only = src.join(tgt, ["account_id", "month_end"], "left_anti")
right_only = tgt.join(src, ["account_id", "month_end"], "left_anti")
```

Real database / enterprise context: Migration validation from Hive to Databricks/Snowflake/Athena.

Interview tip: Do not collect full data to Python for comparison.

Catalogs, Databases and Cloud

Q56. How do you read from PostgreSQL or SQL Server using JDBC?

Real answer: Use Spark JDBC reader with partitionColumn, lowerBound, upperBound and numPartitions for large tables. Pulling via single connection is slow. Use predicates or CDC when possible.

Code / SQL / PySpark example:

```
jdbc_df = spark.read.format("jdbc")
    .option("url", jdbc_url)
    .option("dbtable", "dbo.transactions")
    .option("partitionColumn", "txn_id")
```

```
.option("lowerBound", "1").option("upperBound", "10000000")  
.option("numPartitions", "20").load()
```

Real database / enterprise context: PostgreSQL, SQL Server, Oracle and MySQL source systems.

Interview tip: Mention secrets from Key Vault/Secrets Manager, not hardcoded passwords.

Q57. How do you write PySpark output for Athena querying?

Real answer: Write partitioned Parquet to S3 and update Glue Catalog metadata. Avoid too many small files and ensure compatible data types. Athena works best with Parquet/ORC and partition pruning.

Code / SQL / PySpark example:

```
df.write.mode("append").partitionBy("month_end_date").parquet("s3://lake/gold/txn/")  
# Glue crawler or MSCK REPAIR TABLE in Athena
```

Real database / enterprise context: AWS S3 + Glue + Athena architecture.

Interview tip: Mention S3 path layout and partition projection where suitable.

Q58. How do you use Unity Catalog with PySpark?

Real answer: Use fully qualified names catalog.schema.table, enforce permissions at catalog/schema/table level and avoid unmanaged data access for governed datasets. Unity Catalog improves lineage and security.

Code / SQL / PySpark example:

```
df = spark.table("prod_silver.banking.transactions")  
df.write.saveAsTable("prod_gold.risk.customer_features")
```

Real database / enterprise context: Databricks enterprise data platforms.

Interview tip: Discuss governance, lineage and least privilege access.

Q59. How do you load data from ADLS Gen2 in Azure Databricks?

Real answer: Mounts are older; modern approach uses Unity Catalog external locations or service principals/managed identities. Read data using abfss paths and keep secrets out of code.

Code / SQL / PySpark example:

```
df = spark.read.format("parquet").load("abfss://silver@storage.dfs.core.windows.net/transactions/")
```

Real database / enterprise context: Azure Data Lake Storage Gen2 and Databricks.

Interview tip: Mention RBAC, ACLs and managed identity/service principal.

Q60. How do you choose between Parquet and Delta?

Real answer: Use Parquet for simple immutable analytical files. Use Delta when you need ACID transactions, upserts, deletes, schema enforcement, time travel, streaming consistency or table optimization.

Code / SQL / PySpark example:

```
df.write.format("delta").mode("overwrite").saveAsTable("silver.accounts")
```

Real database / enterprise context: Delta is preferred for curated lakehouse tables; Parquet may suit exports/interchange.

Interview tip: Do not say Delta is always required; explain trade-offs.

Q61. How do you query Spark tables using SQL?

Real answer: Create temp views or save tables to catalog. Spark SQL is often clearer for analysts and complex transformations. In production, choose either DataFrame API or SQL consistently based on team standards.

Code / SQL / PySpark example:

```
df.createOrReplaceTempView("txn")  
spark.sql("SELECT account_id, SUM(amount) FROM txn GROUP BY account_id").show()
```

Real database / enterprise context: Useful in Databricks notebooks, EMR Spark SQL and Synapse Spark.

Interview tip: Both APIs compile to Spark plans; syntax preference is not performance by itself.

Q62. How do you secure database credentials in PySpark jobs?

Real answer: Use secrets management: Databricks secrets, Azure Key Vault, AWS Secrets Manager or environment-injected credentials. Never hardcode usernames/passwords in notebooks or repos.

Code / SQL / PySpark example:

```
password = dbutils.secrets.get(scope="prod", key="sql-password")
```

Real database / enterprise context: Banking environments enforce secret rotation and audit.

Interview tip: Mention service principals, IAM roles and managed identities.

Q63. How do you handle PII data in PySpark?

Real answer: Classify sensitive columns, mask/tokenize where required, restrict access through catalog permissions, encrypt storage and logs, and avoid writing PII to debug outputs. Use column-level controls where available.

Code / SQL / PySpark example:

```
masked = df.withColumn("email_masked", regexp_replace("email", "(^.*?)(@.*$)", "$1***$2"))
```

Real database / enterprise context: GDPR, PCI and banking privacy requirements.

Interview tip: Avoid show() on PII in notebooks.

Advanced PySpark

Q64. What are accumulators and broadcast variables?

Real answer: Broadcast variables distribute read-only data to executors efficiently. Accumulators aggregate values from executors to driver, commonly for counters, but should not drive business logic because retries can affect exact counts.

Code / SQL / PySpark example:

```
bc_rules = spark.sparkContext.broadcast({"HIGH":1000})  
# Use broadcast variable in limited cases; prefer DataFrame broadcast joins for tables.
```

Real database / enterprise context: Reference rules, lookup maps and metrics counters.

Interview tip: Prefer DataFrame broadcast join for structured lookup data.

Q65. How do you use mapPartitions and when?

Real answer: mapPartitions processes one partition at a time and can reduce setup overhead, such as opening one database/API connection per partition instead of per row. Use carefully because it bypasses many DataFrame optimizations.

Code / SQL / PySpark example:

```
def process_partition(rows):  
    for r in rows:  
        yield (r.account_id, r.amount)  
out = df.rdd.mapPartitions(process_partition).toDF(["account_id", "amount"])
```

Real database / enterprise context: Used for custom API/database writes, but often avoided in curated ETL.

Interview tip: Mention DataFrame APIs first; use RDD only when justified.

Q66. What are pandas UDFs and when are they useful?

Real answer: Pandas UDFs use Apache Arrow to vectorize Python operations over batches, faster than standard Python UDFs. They are useful for custom calculations not available as Spark built-ins, but still require careful memory control.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import pandas_udf  
@pandas_udf("double")  
def score(v):  
    return v * 1.2  
df.withColumn("score", score("amount"))
```

Real database / enterprise context: Feature engineering or custom scoring logic.

Interview tip: Built-in Spark functions are still preferred when available.

Q67. How do you handle driver memory errors?

Real answer: Avoid collect(), toPandas() and large broadcast objects. Increase driver memory only after fixing design. Write outputs to storage, sample data for debugging and aggregate before bringing data to driver.

Code / SQL / PySpark example:

```
sample = df.limit(1000).toPandas() # safer than full toPandas  
# Better: write large result instead of collect
```


Real database / enterprise context: Common in notebooks when engineers inspect large DataFrames.

Interview tip: Say driver is coordinator, not data processing engine.

Q68. How do you handle executor out-of-memory errors?

Real answer: Reduce partition size, avoid wide rows, tune executor memory/cores, fix skew, reduce caching, use efficient formats and avoid explosive joins. Check Spark UI for spills and failed tasks.

Code / SQL / PySpark example:

```
spark.conf.set("spark.sql.shuffle.partitions", "400")
# but tune based on data and cluster, not blindly
```

Real database / enterprise context: Large transaction joins and feature generation are common causes.

Interview tip: Do not solve every OOM by increasing memory only.

Q69. How do you prevent Cartesian joins?

Real answer: Ensure join conditions are correct. Spark may create Cartesian products when joining without conditions or when conditions are invalid. Use explicit join keys and validate row counts after joins.

Code / SQL / PySpark example:

```
bad = a.crossJoin(b) # only if intentional
good = a.join(b, "customer_id", "inner")
```

Real database / enterprise context: A missing account_id join condition can multiply banking fact tables massively.

Interview tip: Mention row count sanity checks after joins.

Q70. How do you work with arrays and structs?

Real answer: Use explode, posexplode, inline, getField and higher-order functions. Keep nested structures in bronze; flatten into silver/gold based on analytical need.

Code / SQL / PySpark example:

```
from pyspark.sql.functions import explode
items = orders.select("order_id", explode("items").alias("item"))
items.select("order_id", "item.sku", "item.qty")
```

Real database / enterprise context: API orders, claims line items and event payloads.

Interview tip: Preserve parent keys and array positions when required.

Q71. How do you create reusable transformation functions?

Real answer: Write pure functions that accept and return DataFrames. Keep I/O outside transformation functions. This improves testing, CI/CD and reuse across pipelines.

Code / SQL / PySpark example:

```
def clean_transactions(df):
    return df.filter("txn_id IS NOT NULL").dropDuplicates(["txn_id"])
cleaned = clean_transactions(raw)
```

Real database / enterprise context: Used in production repos rather than ad-hoc notebooks.

Interview tip: This separates business logic from orchestration.

Q72. How do you parameterize PySpark jobs?

Real answer: Pass environment, dates, input paths and table names as job parameters. Avoid hardcoding dev/prod resources. In Databricks use widgets or job parameters; in EMR/Spark submit use arguments.

Code / SQL / PySpark example:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--run_date")
args = parser.parse_args()
```

Real database / enterprise context: Monthly banking batch pipelines and backfills.

Interview tip: Mention config-driven pipelines.

Orchestration and Production

Q73. How do you design an idempotent batch PySpark pipeline?

Real answer: Use deterministic inputs, overwrite specific partitions or MERGE by keys, maintain audit/control tables and avoid append-only duplicates. Rerunning the same batch should produce the same target state.

Code / SQL / PySpark example:

```
df.write.mode("overwrite").option("replaceWhere", "month_end_date = '2025-01-31']").format("delta").saveAsTable("gold.month_h")
```

Real database / enterprise context: Monthly regulatory reporting and financial close pipelines.

Interview tip: Idempotency is a senior-level keyword, but explain it with mechanics.

Q74. How do you perform backfill safely?

Real answer: Backfill by partition/date range, isolate from current production runs, validate counts and control totals, and track run metadata. Avoid full historical reload unless necessary.

Code / SQL / PySpark example:

```
for m in months:
    spark.sql(f"DELETE FROM gold.txn WHERE month_end_date = '{m}'")
    # rebuild that month only
```

Real database / enterprise context: Backfilling corrected transactions from 2022-2025 in banking.

Interview tip: Discuss impact on downstream dashboards and SLAs.

Q75. How do you monitor PySpark jobs?

Real answer: Track job status, runtime, input/output counts, error counts, data quality checks, SLA breaches and cost. Use platform logs plus custom audit tables. Alerts should include business context, not only stack traces.

Code / SQL / PySpark example:

```
audit = [(batch_id, source_count, target_count, status)]
spark.createDataFrame(audit, schema).write.mode("append").saveAsTable("ops.pipeline_audit")
```

Real database / enterprise context: Databricks Jobs, Airflow, ADF, CloudWatch, Azure Monitor.

Interview tip: Good monitoring includes technical and business metrics.

Q76. How do you design retry logic?

Real answer: Retries help transient failures but can create duplicates if the job is not idempotent. Use orchestrator-level retries with idempotent writes and clear failure boundaries.

Code / SQL / PySpark example:

```
# In Airflow/ADF configure retries; in job logic ensure MERGE/partition overwrite is safe
```

Real database / enterprise context: Network, S3 throttling, JDBC timeouts and temporary cluster issues.

Interview tip: Say retry is not a replacement for idempotent design.

Q77. How do you deploy PySpark code using CI/CD?

Real answer: Store code in Git, run unit tests and linting, package jobs, deploy configs per environment and promote through dev/UAT/prod. Avoid manual notebook-only production changes.

Code / SQL / PySpark example:

```
pytest tests/
databricks bundle deploy -t prod
```

Real database / enterprise context: Azure DevOps/GitHub Actions/GitLab CI with Databricks or EMR.

Interview tip: Mention environment-specific configs and approvals.

Q78. How do you handle dependencies in PySpark jobs?

Real answer: Pin library versions, package shared code, use cluster policies or job-specific libraries and avoid installing libraries interactively in production notebooks.

Code / SQL / PySpark example:

```
pip install -r requirements.txt
# or wheel deployment to cluster/job
```

Real database / enterprise context: Databricks wheels, EMR bootstrap actions, Synapse libraries.

Interview tip: Version drift causes production failures.

Q79. How do you handle logging in PySpark?

Real answer: Use structured logging with batch_id, table_name, row counts and error context. Avoid printing sensitive records. Logs should be searchable and integrated with platform monitoring.

Code / SQL / PySpark example:

```
import logging
logger = logging.getLogger("txn_pipeline")
logger.info("batch_id=%s stage=silver_load status=started", batch_id)
```

Real database / enterprise context: Banking audit and incident investigation.

Interview tip: Logging is not just print statements.

Q80. How do you manage dev, test and prod environments?

Real answer: Separate catalogs, storage paths, secrets, clusters and permissions. Use config files or deployment variables to avoid changing code between environments.

Code / SQL / PySpark example:

```
table = f"{env}_silver.banking.transactions"
df.write.saveAsTable(table)
```

Real database / enterprise context: Unity Catalog, Glue Catalog databases, Synapse workspaces.

Interview tip: Mention least privilege and production change controls.

AI/ML Data Engineering with PySpark

Q81. How do you build features for ML using PySpark?

Real answer: Clean source data, join labels, aggregate behavioral features, avoid leakage, write features to a feature store or Delta table, and track feature definitions. PySpark is used because feature datasets can be very large.

Code / SQL / PySpark example:

```
features = txn.groupBy("customer_id").agg(
    count("").alias("txn_30d_count"),
    avg("amount").alias("avg_txn_amount")
)
```

Real database / enterprise context: Credit risk, churn prediction and fraud models.

Interview tip: Mention point-in-time correctness.

Q82. What is point-in-time correctness?

Real answer: Features must only use data available at prediction time. Using future data creates leakage and overstates model performance. This is critical in financial risk and fraud models.

Code / SQL / PySpark example:

```
train = features.join(labels, "customer_id").filter("feature_ts <= label_ts")
```

Real database / enterprise context: Banking default prediction and fraud scoring.

Interview tip: This is a strong ML/data engineering interview answer.

Q83. How do you prepare data for a fraud model?

Real answer: Create transaction velocity, merchant risk, device, channel, geo and customer history features. Handle imbalance, label delay, missing values and streaming requirements. Keep audit trail for feature generation.

Code / SQL / PySpark example:

```
fraud_features = txn.groupBy("account_id").agg(
    sum(when(col("amount") > 1000, 1).otherwise(0)).alias("high_value_count")
)
```

Real database / enterprise context: Card fraud, payment fraud and AML monitoring.

Interview tip: Do not focus only on model; data quality drives model quality.

Q84. How do you monitor ML feature drift with PySpark?

Real answer: Compare current feature distributions with training baseline using metrics such as mean, percentiles, PSI or KL divergence. Trigger alerts when drift crosses thresholds.

Code / SQL / PySpark example:

```
current = df.agg(avg("avg_txn_amount"), expr("percentile(avg_txn_amount, 0.5)"))
```

Real database / enterprise context: Risk and fraud models degrade when customer behavior changes.

Interview tip: Mention drift, data quality and model performance separately.

Q85. How do you create training and inference datasets?

Real answer: Training data includes historical labels and point-in-time features. Inference data includes latest features without labels. Both should share the same transformation logic to avoid training-serving skew.

Code / SQL / PySpark example:

```
train_df = features.join(labels, ["customer_id", "as_of_date"])
inference_df = latest_features.filter("as_of_date = current_date()")
```

Real database / enterprise context: Customer churn, credit risk and claim fraud models.

Interview tip: Reusable pipelines reduce training-serving skew.

Q86. How would you build a RAG document preprocessing pipeline in PySpark?

Real answer: Use PySpark to ingest documents/metadata, clean text, chunk documents, generate embeddings using a model service, and write chunks plus embeddings to a vector database. Track document version and source lineage.

Code / SQL / PySpark example:

```
docs = spark.read.text("/mnt/docs")
# clean/chunk in Spark; call embedding service in controlled batches
```

Real database / enterprise context: Enterprise knowledge search for policies, procedures and contracts.

Interview tip: Mention access control and PII filtering before embeddings.

Q87. How do you evaluate data quality before feeding an ML model?

Real answer: Check missingness, duplicate keys, outliers, feature freshness, label distribution and referential integrity. Fail pipelines if critical model inputs are broken.

Code / SQL / PySpark example:

```
dq = features.agg(
    sum(col("income").isNull().cast("int")).alias("missing_income"),
    countDistinct("customer_id").alias("customers")
)
```

Real database / enterprise context: Banking models require governance around input features.

Interview tip: Model accuracy starts with reliable data pipelines.

Q88. How do you store ML-ready features?

Real answer: Use Delta tables or a feature store with primary keys, timestamps, descriptions, owners and freshness SLAs. Features must be discoverable and reusable across models.

Code / SQL / PySpark example:

```
features.write.format("delta").mode("append").saveAsTable("ml_features.customer_risk_daily")
```

Real database / enterprise context: Databricks Feature Store, Feast, SageMaker Feature Store.

Interview tip: Discuss offline and online feature serving if relevant.

Interview Scenarios

Q89. A PySpark job reads 5TB and runs for 6 hours. What do you check first?

Real answer: Check partition pruning, file format, filters, selected columns, shuffle stages, skew, cluster sizing, small files and join strategy. Start with Spark UI and explain plan. Then optimize the biggest bottleneck rather than randomly changing configs.

Code / SQL / PySpark example:

```
df.filter("business_date = '2025-01-31'").select("account_id", "amount")
# Verify partition pruning in explain plan
```

Real database / enterprise context: Large monthly transaction processing in banks.

Interview tip: This is a troubleshooting question; answer with a diagnostic sequence.

Q90. You have duplicates after a retry. How do you fix the pipeline?

Real answer: Make writes idempotent using partition overwrite or MERGE by natural key. Add batch audit and file tracking. Deduplicate source by business key and latest ingestion timestamp before writing.

Code / SQL / PySpark example:

```
dedup = df.withColumn("rn", row_number().over(Window.partitionBy("txn_id").orderBy(col("ingestion_ts").desc()))).filter("rn = 1")
```

Real database / enterprise context: Retried ADF/Airflow/Databricks jobs often cause duplicate appends.

Interview tip: Mention preventing future issue, not only cleaning current data.

Q91. A join increases row count unexpectedly. What happened?

Real answer: Most likely duplicate keys on one or both sides caused many-to-many join explosion, or join condition is incomplete. Check key uniqueness and join grain before joining.

Code / SQL / PySpark example:

```
df.groupBy("account_id").count().filter("count > 1").show()
```

Real database / enterprise context: Joining transactions to a non-unique customer/account dimension.

Interview tip: Always validate grain of each dataset.

Q92. How do you process 10 years of historical data?

Real answer: Process by partition/date range, parallelize where safe, validate each batch, write to partitioned Delta/Parquet and maintain audit logs. Avoid one monolithic job. Reuse same logic as incremental loads where possible.

Code / SQL / PySpark example:

```
for y in range(2015, 2026):
    process_year(y)
```

Real database / enterprise context: Historical migration from Hadoop/Hive to Databricks/AWS.

Interview tip: Mention backfill strategy and downstream cutover.

Q93. How do you handle schema drift from Kafka JSON events?

Real answer: In bronze, capture raw payload and inferred/declared schema version. In silver, parse expected fields, route unknown/bad records to quarantine and update schema contract through controlled process.

Code / SQL / PySpark example:

```
raw.selectExpr("CAST(value AS STRING) as json_payload")
# parse with from_json using versioned schema
```

Real database / enterprise context: Digital events, mobile app logs and payment events.

Interview tip: Never let ungoverned schema drift break gold tables.

Q94. How would you migrate a Hive ETL job to PySpark/Delta?

Real answer: Understand source logic, table grain, partitions and downstream users. Rebuild transformations in PySpark/SQL, validate row counts/control totals, run parallel testing and cut over with rollback plan.

Code / SQL / PySpark example:

```
legacy = spark.table("hive_db.txn")
new = transform(legacy)
new.write.format("delta").saveAsTable("silver.txn")
```

Real database / enterprise context: Hadoop modernization to Databricks, EMR or Synapse.

Interview tip: Migration is about correctness, not just rewriting code.

Q95. How do you deal with null keys in joins?

Real answer: Null join keys do not match in standard equality joins. Decide business rule: reject, default, keep unmatched, or use null-safe equality. For key fields like account_id, nulls usually go to exception handling.

Code / SQL / PySpark example:

```
valid = df.filter("account_id IS NOT NULL")
invalid = df.filter("account_id IS NULL")
```

Real database / enterprise context: Source data quality issue in customer/account feeds.

Interview tip: Do not silently coalesce null keys to a fake value unless approved.

Q96. How do you calculate latest status per account?

Real answer: Use row_number over account_id ordered by status timestamp descending and ingestion timestamp descending. This handles multiple updates per account and late arrivals.

Code / SQL / PySpark example:

```
w = Window.partitionBy("account_id").orderBy(col("status_ts").desc(), col("ingestion_ts").desc())
latest = df.withColumn("rn", row_number().over(w)).filter("rn=1")
```

Real database / enterprise context: Account lifecycle, loan status and insolvency status feeds.

Interview tip: Clarify ordering column when timestamps tie.

Q97. How do you implement delete handling in CDC?

Real answer: Read CDC operation codes. For deletes, either physically delete from target using Delta DELETE/MERGE or mark records inactive based on business/audit requirements. Regulated systems often prefer soft deletes.

Code / SQL / PySpark example:

```
MERGE INTO silver.accounts t USING cdc s ON t.account_id=s.account_id
WHEN MATCHED AND s.op = 'D' THEN UPDATE SET t.is_deleted = true
```

Real database / enterprise context: SQL Server/Oracle CDC to lakehouse.

Interview tip: Ask whether hard delete is allowed by retention policy.

Q98. How do you ensure exactly-once processing?

Real answer: Exactly-once depends on source, sink and idempotent logic. Use checkpoints, transactional sinks like Delta, deterministic keys and MERGE/upsert. Avoid non-idempotent external side effects inside streaming jobs.

Code / SQL / PySpark example:

```
stream.writeStream.format("delta").option("checkpointLocation", chk).start(target)
```

Real database / enterprise context: Payment and transaction streams.

Interview tip: Be precise: Spark can provide end-to-end exactly-once only with compatible sources/sinks.

Q99. How do you choose cluster size for a PySpark job?

Real answer: Estimate input size, transformation complexity, shuffle volume, SLA and cost. Start with workload metrics, not guesswork. Scale out for parallelism, scale up for memory-heavy joins, and use autoscaling cautiously for predictable jobs.

Code / SQL / PySpark example:

```
# Review input size, partitions, shuffle, executor CPU/memory in Spark UI
```

Real database / enterprise context: Large nightly batches vs near-real-time streaming need different sizing.

Interview tip: Mention cost-performance trade-off.

Q100. What would you do if a job succeeds but output count is wrong?

Real answer: Check source filters, joins, dedup logic, partition overwrite scope, late data, schema parsing and business rules. Compare intermediate counts at each stage using audit checkpoints.

Code / SQL / PySpark example:

```
audit(stage="bronze", count=bronze.count())
audit(stage="silver", count=silver.count())
```

Real database / enterprise context: Production incident after release.

Interview tip: Strong candidates propose observability checkpoints.

Final Interview Checklist

Area	What to be ready to explain
Spark basics	Driver, executors, partitions, lazy evaluation, DAG, actions vs transformations
Performance	Shuffle, skew, broadcast joins, partition tuning, caching, predicate pushdown
Lakehouse	Delta MERGE, SCD2, schema evolution, time travel, OPTIMIZE, VACUUM
Streaming	Kafka, checkpointing, watermarking, foreachBatch, idempotency
Production	CI/CD, secrets, monitoring, audit tables, retries, backfill, data quality
Business context	Banking transaction pipelines, customer 360, fraud, risk, reconciliation